Varying Intercepts Models

General Principles

To model the relationship between a dependent variable and an independent variable while allowing for different intercepts across groups or clusters, we can use a *Varying Intercepts* model. This approach is particularly useful when data are grouped (e.g., by subject, location, or time period) and we expect the baseline level of the outcome to vary across these groups.

Considerations

> **Note**
>
> - We have the same considerations as for Regression for a continuous variable.
> - The main idea of varying intercepts is to generate an intercept for each group, allowing each group to start at different levels. Thus, the intercept α_k is defined uniquely for each of the K declared groups.
> - In the code below, the intercept `alpha` for each of the k declared groups shares two priors, `a_bar` and `sigma`, which are respectively modeled by a Normal and an Exponential distribution.

Example

Below is an example code snippet demonstrating Bayesian regression with varying intercepts using the BayesForge (BF) package. The data consists of a dependent variable representing individuals' survival (*surv*) and an independent categorical variable (*tank*), which indicates the tank where the individual was born, with a total of 48 tanks. This example is based on McElreath (2018).

Python (Raw)

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device-----------------------------------------------
m = bf(platform='cpu')

# Import Data & Data Manipulation ----------------------------------------------
# Import
from importlib.resources import files
data_path = m.load.reedfrogs(only_path=True)
m.data(data_path, sep=';')
# Manipulate
m.df["tank"] = jnp.arange(m.df.shape[0])

# Define model -----------------------------------------------
def model(tank, surv, density):
    sigma = m.dist.exponential( 1,  name = 'sigma')
    a_bar = m.dist.normal( 0., 1.5,  name = 'a_bar')
    alpha = m.dist.normal( a_bar, sigma, shape= tank.shape, name = 'alpha')
    p = alpha[tank]
    m.dist.binomial(total_count = density, logits = p, obs=surv)

# Run sampler -----------------------------------------------
m.fit(model, progress_bar=False)

# Diagnostic -----------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a_bar	1.34	0.26	0.94	1.77	3432.60	2881.04	1.0	0.00	0.00
alpha[0]	2.13	0.86	0.88	3.62	4634.17	2871.30	1.0	0.01	0.01
alpha[1]	3.07	1.12	1.46	4.99	4505.47	2687.65	1.0	0.02	0.02
alpha[2]	1.00	0.67	-0.03	2.10	4084.55	2991.56	1.0	0.01	0.01

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha[3]	3.05	1.11	1.48	4.98	5068.53	2562.56	1.0	0.02	0.02
alpha[4]	2.15	0.88	0.87	3.68	4412.87	2885.83	1.0	0.01	0.02
alpha[5]	2.13	0.88	0.88	3.60	4809.87	2604.83	1.0	0.01	0.02
alpha[6]	3.05	1.11	1.47	4.94	3932.43	2367.67	1.0	0.02	0.02
alpha[7]	2.13	0.89	0.88	3.61	4667.52	2599.12	1.0	0.01	0.02
alpha[8]	-0.18	0.60	-1.14	0.81	4540.87	2903.94	1.0	0.01	0.01
alpha[9]	2.14	0.89	0.84	3.61	4740.09	2667.42	1.0	0.01	0.02
alpha[10]	1.01	0.67	-0.02	2.11	4136.03	2912.36	1.0	0.01	0.01
alpha[11]	0.58	0.61	-0.39	1.59	5385.63	2855.28	1.0	0.01	0.01
alpha[12]	1.00	0.67	-0.03	2.10	5338.33	2734.52	1.0	0.01	0.01
alpha[13]	0.20	0.61	-0.79	1.18	5144.15	3177.17	1.0	0.01	0.01
alpha[14]	2.14	0.87	0.87	3.62	4549.45	2562.34	1.0	0.01	0.02
alpha[15]	2.15	0.86	0.85	3.62	5265.69	2990.94	1.0	0.01	0.01
alpha[16]	2.89	0.76	1.79	4.19	5238.60	2831.52	1.0	0.01	0.01
alpha[17]	2.40	0.67	1.41	3.54	4975.10	2783.89	1.0	0.01	0.01
alpha[18]	2.02	0.59	1.15	3.02	4905.99	2681.67	1.0	0.01	0.01
alpha[19]	3.62	0.98	2.24	5.30	4385.66	2563.51	1.0	0.02	0.02
alpha[20]	2.39	0.68	1.39	3.56	4613.16	2680.88	1.0	0.01	0.01
alpha[21]	2.38	0.65	1.43	3.47	4498.35	2622.99	1.0	0.01	0.01
alpha[22]	2.40	0.66	1.42	3.54	5193.48	2546.75	1.0	0.01	0.01
alpha[23]	1.69	0.52	0.90	2.55	4442.09	3077.24	1.0	0.01	0.01
alpha[24]	-1.00	0.45	-1.74	-0.30	5145.60	2822.13	1.0	0.01	0.01
alpha[25]	0.16	0.39	-0.47	0.78	4352.57	2839.08	1.0	0.01	0.01
alpha[26]	-1.44	0.50	-2.28	-0.69	4610.86	2878.60	1.0	0.01	0.01
alpha[27]	-0.48	0.41	-1.13	0.17	4421.54	2891.35	1.0	0.01	0.01
alpha[28]	0.17	0.40	-0.46	0.79	5049.23	3157.41	1.0	0.01	0.01
alpha[29]	1.45	0.48	0.72	2.25	5068.90	3027.66	1.0	0.01	0.01
alpha[30]	-0.64	0.41	-1.28	0.02	4911.65	3051.21	1.0	0.01	0.01
alpha[31]	-0.32	0.41	-0.98	0.33	4048.72	2773.13	1.0	0.01	0.01
alpha[32]	3.20	0.77	2.08	4.48	4353.53	2798.03	1.0	0.01	0.01
alpha[33]	2.69	0.63	1.77	3.76	4720.67	2815.40	1.0	0.01	0.01
alpha[34]	2.69	0.63	1.76	3.80	5079.81	2451.50	1.0	0.01	0.01
alpha[35]	2.06	0.52	1.29	2.93	4693.79	2643.71	1.0	0.01	0.01
alpha[36]	2.06	0.51	1.28	2.93	5309.52	2490.38	1.0	0.01	0.01
alpha[37]	3.91	0.99	2.53	5.63	3148.58	2342.31	1.0	0.02	0.02
alpha[38]	2.70	0.63	1.78	3.78	4491.32	2895.61	1.0	0.01	0.01
alpha[39]	2.34	0.54	1.53	3.24	5439.71	2836.48	1.0	0.01	0.01
alpha[40]	-1.81	0.47	-2.60	-1.10	5453.63	3085.04	1.0	0.01	0.01
alpha[41]	-0.57	0.35	-1.14	-0.03	4046.15	2677.36	1.0	0.01	0.01
alpha[42]	-0.45	0.35	-1.03	0.09	4134.51	2772.07	1.0	0.01	0.01
alpha[43]	-0.33	0.34	-0.88	0.20	4898.42	2821.90	1.0	0.00	0.01

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha[44]	0.58	0.34	0.05	1.11	5004.97	2920.80	1.0	0.00	0.01
alpha[45]	-0.57	0.35	-1.15	-0.04	4556.75	2975.00	1.0	0.01	0.01
alpha[46]	2.07	0.50	1.31	2.89	4636.72	3115.95	1.0	0.01	0.01
alpha[47]	0.00	0.34	-0.54	0.54	4902.83	3158.32	1.0	0.00	0.01
sigma	1.62	0.22	1.32	1.98	2399.85	2779.66	1.0	0.00	0.00

Python (Build in function)

```
from BayesForge import bf
import jax.numpy as jnp


# Setup device--------------------------------------------------
m = bf(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
# Import
from importlib.resources import files
data_path =  m.load.reedfrogs(only_path=True)
m.data(data_path, sep=';')
# Manipulate
m.df["tank"] = jnp.arange(m.df.shape[0])

# Define model -------------------------------------------------
def model(tank, surv, density):
    alpha = m.effects.varying_intercept(N_groups=48,group_id=tank, group_name = 'tank')
    m.dist.binomial(total_count = density, logits = alpha, obs=surv)

# Run sampler --------------------------------------------------
m.fit(model, progress_bar=False)

# Diagnostic ---------------------------------------------------
m.summary()
```

```
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcs
global_intercept_tank	1.36	0.26	0.95	1.78	4075.56	3145.70	1.0	0.00	0.00
intercept_tank[0]	2.13	0.87	0.86	3.61	5418.48	2837.07	1.0	0.01	0.02
intercept_tank[1]	3.06	1.10	1.45	4.97	4290.46	2614.05	1.0	0.02	0.02
intercept_tank[2]	1.00	0.68	-0.04	2.17	5406.15	2660.66	1.0	0.01	0.01
intercept_tank[3]	3.04	1.08	1.49	4.94	5359.23	2823.08	1.0	0.02	0.02
intercept_tank[4]	2.13	0.87	0.86	3.58	4493.42	2562.70	1.0	0.01	0.02
intercept_tank[5]	2.13	0.89	0.82	3.66	5376.96	2638.05	1.0	0.01	0.02
intercept_tank[6]	3.06	1.11	1.45	4.95	5201.56	2641.48	1.0	0.02	0.02
intercept_tank[7]	2.14	0.86	0.85	3.55	5108.89	2789.61	1.0	0.01	0.01
intercept_tank[8]	-0.16	0.61	-1.14	0.82	5026.04	2940.30	1.0	0.01	0.01
intercept_tank[9]	2.13	0.89	0.81	3.61	5471.72	2927.84	1.0	0.01	0.02
intercept_tank[10]	0.99	0.67	-0.02	2.11	5886.74	2750.11	1.0	0.01	0.01
intercept_tank[11]	0.59	0.63	-0.40	1.62	5803.64	2923.97	1.0	0.01	0.01
intercept_tank[12]	1.01	0.68	-0.06	2.10	5476.72	2803.17	1.0	0.01	0.01
intercept_tank[13]	0.19	0.62	-0.81	1.20	6118.22	3067.40	1.0	0.01	0.01
intercept_tank[14]	2.17	0.87	0.91	3.66	4187.34	2536.46	1.0	0.01	0.02
intercept_tank[15]	2.13	0.86	0.87	3.59	5939.79	2667.54	1.0	0.01	0.01
intercept_tank[16]	2.90	0.77	1.80	4.20	5142.35	2668.76	1.0	0.01	0.01
intercept_tank[17]	2.41	0.69	1.41	3.57	5141.07	2586.88	1.0	0.01	0.01
intercept_tank[18]	2.02	0.58	1.16	3.01	5217.48	2775.14	1.0	0.01	0.01
intercept_tank[19]	3.66	1.00	2.25	5.40	4811.99	2325.02	1.0	0.02	0.02
intercept_tank[20]	2.39	0.66	1.41	3.50	5179.49	2766.81	1.0	0.01	0.01
intercept_tank[21]	2.39	0.66	1.42	3.49	5362.57	2904.22	1.0	0.01	0.01
intercept_tank[22]	2.41	0.66	1.42	3.54	5735.72	2720.77	1.0	0.01	0.01
intercept_tank[23]	1.70	0.52	0.92	2.57	5725.10	3043.28	1.0	0.01	0.01
intercept_tank[24]	-1.01	0.45	-1.75	-0.32	5854.01	2580.80	1.0	0.01	0.01
intercept_tank[25]	0.16	0.39	-0.44	0.78	5449.15	3184.44	1.0	0.01	0.01
intercept_tank[26]	-1.43	0.49	-2.25	-0.68	5269.30	2955.08	1.0	0.01	0.01
intercept_tank[27]	-0.47	0.41	-1.13	0.19	5409.89	2970.45	1.0	0.01	0.01
intercept_tank[28]	0.16	0.40	-0.48	0.81	6564.27	2956.08	1.0	0.00	0.01
intercept_tank[29]	1.44	0.51	0.68	2.29	5458.95	2889.28	1.0	0.01	0.01
intercept_tank[30]	-0.65	0.42	-1.34	0.01	5876.16	3029.47	1.0	0.01	0.01
intercept_tank[31]	-0.30	0.40	-0.95	0.32	5981.66	3291.59	1.0	0.01	0.01
intercept_tank[32]	3.19	0.76	2.09	4.48	5119.38	2659.39	1.0	0.01	0.01
intercept_tank[33]	2.72	0.65	1.77	3.85	4333.09	2353.99	1.0	0.01	0.01
intercept_tank[34]	2.71	0.66	1.75	3.84	5146.90	2558.53	1.0	0.01	0.01
intercept_tank[35]	2.06	0.51	1.30	2.90	5366.45	2928.43	1.0	0.01	0.01
intercept_tank[36]	2.06	0.51	1.29	2.93	5685.59	2727.51	1.0	0.01	0.01
intercept_tank[37]	3.87	0.92	2.52	5.46	4153.86	2544.92	1.0	0.02	0.02
intercept_tank[38]	2.72	0.65	1.77	3.81	5348.39	1891.25	1.0	0.01	0.01
intercept_tank[39]	2.35	0.56	1.53	3.30	5526.31	2928.28	1.0	0.01	0.01

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcs
intercept_tank[40]	-1.81	0.47	-2.59	-1.10	5875.45	3089.18	1.0	0.01	0.01
intercept_tank[41]	-0.57	0.35	-1.14	-0.01	5629.86	3093.80	1.0	0.00	0.01
intercept_tank[42]	-0.45	0.34	-1.00	0.11	6062.16	3031.79	1.0	0.00	0.01
intercept_tank[43]	-0.34	0.34	-0.88	0.19	6132.44	2923.22	1.0	0.00	0.01
intercept_tank[44]	0.58	0.35	0.02	1.15	5481.71	3006.07	1.0	0.00	0.01
intercept_tank[45]	-0.57	0.35	-1.13	-0.01	4917.45	3129.40	1.0	0.01	0.01
intercept_tank[46]	2.06	0.51	1.28	2.92	5247.04	2904.55	1.0	0.01	0.01
intercept_tank[47]	0.01	0.34	-0.54	0.56	5961.61	3030.61	1.0	0.00	0.01
sd_tank	1.61	0.21	1.31	1.97	2447.66	2558.35	1.0	0.00	0.00

R

```r
library(BayesForge)

# setup platform-----------------------------------------------
m=importBF(platform='cpu')

# Import data -------------------------------------------------
m$data(m$load$reedfrogs(only_path=T), sep=';')
m$df$tank = as.integer(0:(nrow(m$df)-1)) # Manipulate
m$data_to_model(list('tank', 'surv', 'density')) # Manipulate
m$data_on_model$tank = m$data_on_model$tank$astype(jnp$int32) # Manipulate
m$data_on_model$surv = m$data_on_model$surv$astype(jnp$int32) # Manipulate
m$data_on_model$density = m$data_on_model$density$astype(jnp$int64) # Manipulate


# Define model ------------------------------------------------
model <- function(tank, surv, density){
  # Parameter prior distributions
  sigma = m$dist$exponential( 1, name = 'sigma')
  a_bar =  m$dist$normal(0, 1.5, name = 'a_bar')
  alpha = m$dist$normal(a_bar, sigma, name = 'alpha', shape = reticulate::tuple(48L))
  p = alpha$`__getitem__`(tank)
  # Likelihood
  m$dist$binomial(total_count = density, logits = p, obs = surv, name = 'surv')
}

# Run MCMC ----------------------------------------------------
m$fit(model) # Optimize model parameters through MCMC sampling
```

```
# Summary ----------------------------------------------
m$summary() # Get posterior distribution
```

Julia

```
using PyBayesForge

# Setup device-----------------------------------------------
m = importBF(platform="cpu")

# Import Data & Data Manipulation ----------------------------------------------
# Import
data_path = m.load.reedfrogs(only_path = true)
m.data(data_path, sep=';')
m.df["tank"] = jnp.arange(m.df.shape[0])

# Define model -----------------------------------------------
@BF function model(tank, surv, density)
    alpha = m.effects.varying_intercept(N_groups=48,group_id=tank, group_name = "tank")
    m.dist.binomial(total_count = density, logits = alpha, obs=surv)
end

# Run mcmc -----------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary ----------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

We model the relationship between the independent variable X and the outcome variable Y while accounting for varying intercepts α for each group where $k(i)$ give us group belonging for observation i, using the following equation:

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$
$$\mu_i = \alpha_{[k(i)]} + \beta X_i$$

$$\beta \sim \text{Normal}(0, 1)$$

$$\sigma \sim \text{Exponential}(1)$$

$$\alpha_{[k]} \sim \text{Normal}(\bar{\alpha}, \varsigma)$$

$$\bar{\alpha} \sim \text{Normal}(0, 1)$$
$$\varsigma \sim \text{Exponential}(1)$$

Where:

- Y_i is the outcome variable for observation i.
- $\alpha_{[k(i)]}$ is the varying intercept corresponding to the group k of observation i.
- β is the regression coefficient.
- σ is a standard deviation parameter, which here has an Exponential prior that constrains it to be positive.
- $\bar{\alpha}$ is the overall mean intercept.
- ς is the variance of the intercepts across groups.

Model validation tests

- Comparison with Stan: `11.Varying intercepts.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence).
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> - We can apply multiple variables similarly to Multiple continuous variables.
> - We can apply interaction terms similarly to Chapter 3.
> - We can apply categorical variables similarly to Chapter 4.

- We can apply varying intercepts with any distribution developed in previous chapters.

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.